Reviewer Pack — Minimal Order of Formal Language Automorphism Groups and Cir…

Entry af2c169a0f02 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 07:55:53 UTC

Minimal Order of Formal Language Automorphism Groups and Circuit Monotone Width

Entry ID: `af2c169a0f02` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 07:55:53 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory (Automata) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every regular language L, the minimal order of an automorphism group of L is linearly related to its corresponding circuit monotone width w_L, such that ord(L) = Θ(w_L).

Rationale (proposer's reasoning):

> Formal language automorphism groups capture symmetries in the structure of languages, which might be reflected in the complexity of circuits recognizing them. The minimal order provides a measure of symmetry strength, potentially exposing non-trivial relationships with circuit complexity.

Taxonomy category: `automata_circuit_monotone_width` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3af20edc454e0f78`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all regular languages L, the ratio ord(L) / w_L falls within the interval [0.5, 2] with a p-value ≤ 0.05 when correlated over 30 random seeds. The conjecture is falsified if this ratio exceeds [2, 4] or is below [0, 0.5], or if the p-value > 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `automorphism groups formal languages AND circuit monotone width` - `regular language automorphism order related to circuit monotone width` - `linear relationship between minimal order of automorphism group and circuit monotone width`

Top relevant hits considered: - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/2104.14760v4] Automorphism and outer automorphism groups of Right-angled Artin groups are not relatively hyperbolic - [http://arxiv.org/abs/1010.3612v2] Explicit examples of equivalence relations and factors with prescribed fundamental group and outer automorphism group - [http://arxiv.org/abs/2503.21676v2] How do language models learn facts? Dynamics, curricula and hallucinations - [http://arxiv.org/abs/1610.00031v1] Discriminating Similar Languages: Evaluations and Explorations - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1509.06670v2] Automorphism Groups and Invariant Theory on PN - [http://arxiv.org/abs/math/0608534v2] On Automorphisms of Finite p-groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_automaton(n):
        # Generate a random automaton with n states
        states = list(range(n))
        alphabet = ['a', 'b']
        transition_table = {q: {} for q in states}
        start_state = 0
        accepting_states = [n-1]

        for q in states:
            for a in alphabet:
                next_q = random.choice(states)
                while next_q == q:
                    next_q = random.choice(states)
                transition_table[q][a] = next_q

        return {
            'states': states,
```

```
        'alphabet': alphabet,
        'transition_table': transition_table,
        'start_state': start_state,
        'accepting_states': accepting_states
    }

def compute_monotone_width(automaton):
    # Compute the monotone width of the automaton
    n = len(automaton['states'])
    width = 0

    for q in automaton['states']:
        if q in automaton['accepting_states']:
            continue

        reachable_states = {q}
        while True:
            new_reachable_states = set()
            for a in automaton['alphabet']:
                next_q = automaton['transition_table'][q][a]
                if next_q not in reachable_states:
                    new_reachable_states.add(next_q)
            if not new_reachable_states:
                break
            reachable_states.update(new_reachable_states)

        width = max(width, len(reachable_states))

    return width

def compute_automorphism_group(automaton):
    # Compute the automorphism group of the automaton
    states = automaton['states']
    n = len(states)
    automorphisms = []

    for perm in itertools.permutations(states):
        is_automorphism = True
        for q in states:
            for a in automaton['alphabet']:
                next_q = automaton['transition_table'][q][a]
                if automaton['transition_table'][perm[q]][a] != perm[next_q]:
                    is_automorphism = False
                    break
            if not is_automorphism:
                break

        if is_automorphism:
            automorphisms.append(perm)

    return len(automorphisms)

n_values = [5, 10, 15, 20, 30, 40]
ord_L_values = []
w_L_values = []
```

```
    for n in n_values:
        automaton = generate_automaton(n)
        ord_L = compute_automorphism_group(automaton)
        w_L = compute_monotone_width(automaton)

        ord_L_values.append(ord_L)
        w_L_values.append(w_L)

    if not ord_L_values or not w_L_values:
        return {
            "metric_name": "ord(L) / w_L",
            "metric_value": None,
            "instances_tested": len(n_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    ratio = sum(ord_L_values[i] / w_L_values[i] for i in range(len(n_values))) / len(n_values)
    if not (0.5 <= ratio <= 2):
        return {
            "metric_name": "ord(L) / w_L",
            "metric_value": ratio,
            "instances_tested": len(n_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": f"Ratio {ratio} outside [0.5, 2]"
        }

    return {
        "metric_name": "ord(L) / w_L",
        "metric_value": ratio,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result['conjecture_holds'] for result in results):
        mean_ratio = sum(result['metric_value'] for result in results) / len(results)
        std_dev = math.sqrt(sum((result['metric_value'] - mean_ratio) ** 2 for result in results)
        support_fraction = len([result for result in results if result['conjecture_holds']]) / len
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fracti
    elif any(result['metric_value'] > 2 or result['metric_value'] < 0.5 for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result['metric_va
        print(f"RESULT: FALSIFIED counterexample=\"Ratio outside [0.5, 2]\" first_failing_seed={fi
```

```
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its intended analysis to determine if the conjecture is supported or falsified. | next: Run the test again with increased time limits and ensure that it completes without crashing. If the test passes, re-evaluate the results based on the pre-registered support conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16049
2	preregistration	ollama_remote	glm4:latest	0	0	10139
3	novelty	ollama_remote	glm4:latest	0	0	10475
4	novelty	ollama_remote	glm4:latest	0	0	9632
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25386
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13222
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11937
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13844
9	judge	ollama_remote	glm4:latest	0	0	11882

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122566 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/af2c169a0f02.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/af2c169a0f02.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/af2c169a0f02.tar.gz` (if generated)